LAB / INTRODUCTION

1. OVERVIEW

1.1 Goal

The purpose of this laboratory is to use the input/outputs of an embedded system.
You need to select the first created project and we will modify it to match the requested things of this new laboratory.

2. EXTENSION BOARD SETUP ON CUBEMX

Now, we want to use the leds and buttons placed on the extension board. For that, we need to know where are placed these input / outputs.

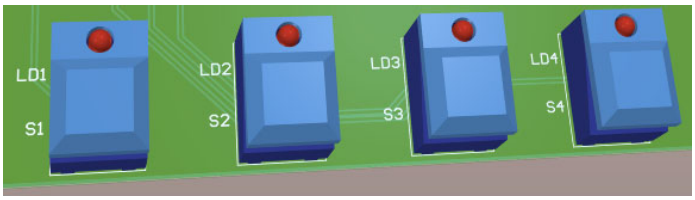


Figure 3: The extension buttons and leds

Input / output	Micocontroller pin
LD1 (led ouput)	PB6
LD2 (led ouput)	PA0
LD3 (led ouput)	PB0
LD4 (led ouput)	PB13
S1 (button input)	PB15
S2 (button input)	PB14
S3 (button input)	PA8
S4 (button input)	PB10

Now, to set up these input/outputs, for each controller pin, we must select the pin on the CPU graphical view and select the mode we want. You must change the I/O name too with the right mouse button.

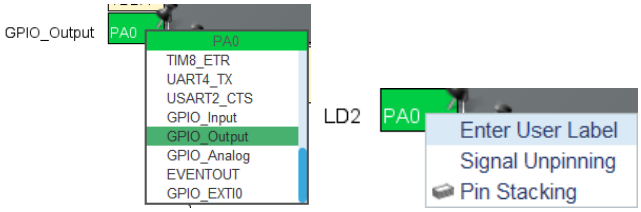


Figure 3: The GPIO setup example

Once this job is done for all I/O pins, you must select the GPIO view one each changed pin to set up particular configuration.

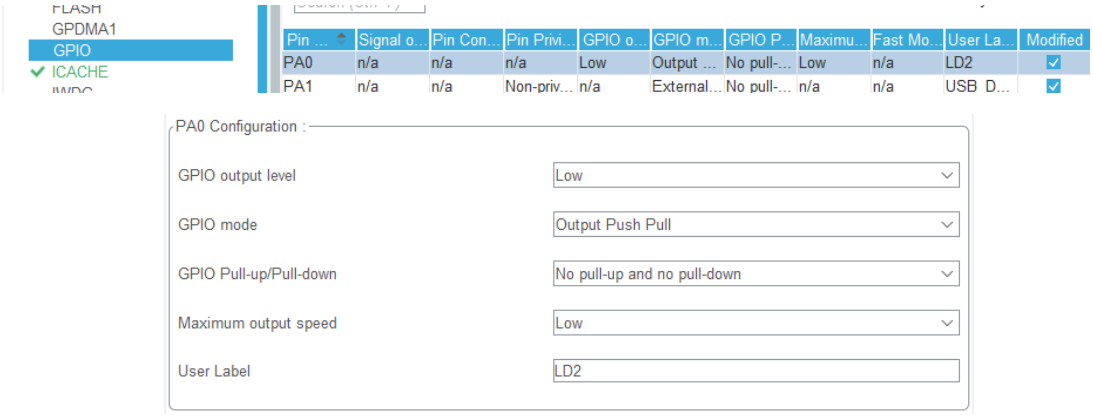
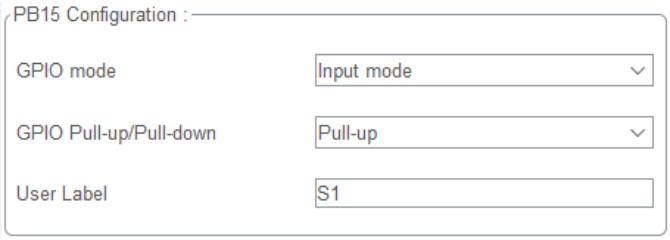


Figure 3: The GPIO configuration

For the outputs, we don't need to change anything. For the inputs, because buttons are active low, we **must activate the pull-up**.



Finally, generate code with the appropriate button.



6. TASK ONE

We want to blink the led LD1. To be able to do this job, we must use some functions of the “Hardware Abstraction Layer” explained below.

To write something on a pin you must use **HAL_GPIO_WritePin**.

```
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, GPIO_PinState
PinState)
```

Remember, you have set up a name for each led. Now, you can use these names as parameters defined in `main.h`:

```
#define S1_Pin         GPIO_PIN_15
#define S1_GPIO_Port   GPIOB
```

The PinState can take two values:

- GPIO_PIN_RESET
- GPIO_PIN_SET

To read something on a pin you must use **HAL_GPIO_ReadPin**.

```
GPIO_PinState HAL_GPIO_ReadPin(const GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
```

Finally, to wait a moment, you can use **HAL_Delay**. By default, the delay is in milliseconds.

```
__weak void HAL_Delay(uint32_t Delay)
```

If you have a look at the file `stm32u5xx_hal_gpio.c`, you can find another function that best match the job we must do.

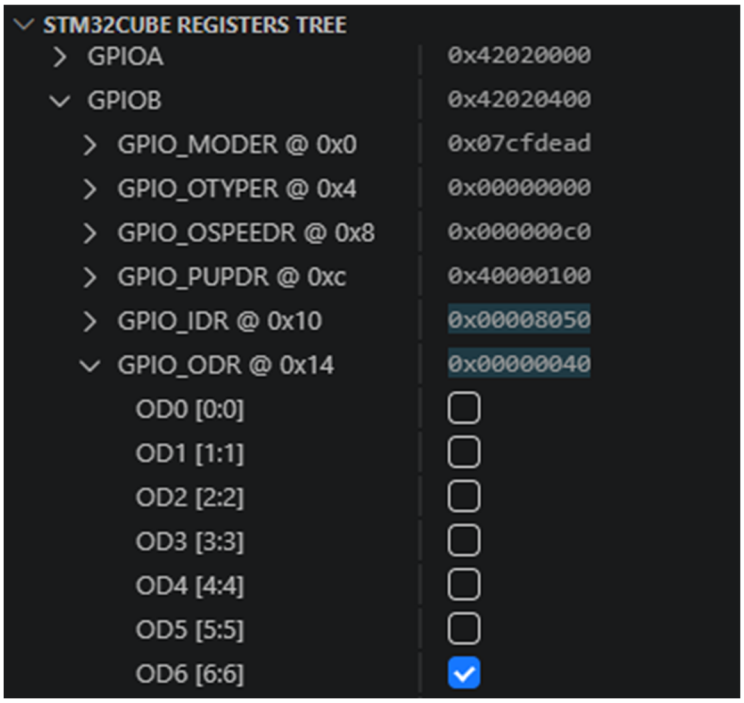
Caution -> TODO absolutely

Because of a silicium errata in this microcontroller, you must include the code below in your initialization part. The line in bold is enough.

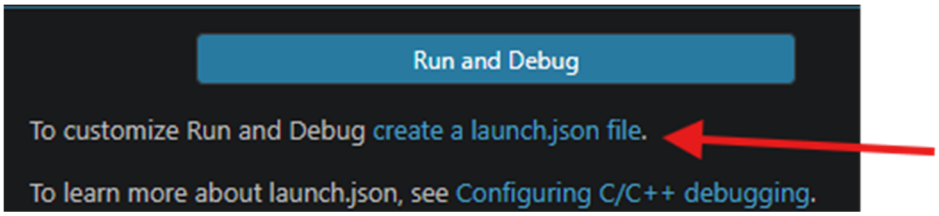
```
#ifdef STM32U5G9xx
    // Fix for BUTTON0 (PB15) to work correctly (otherwise pull-up has no effect)
    //  See:  https://community.st.com/t5/stm32-mcus-products/stm32u5a5-pb5-and-
pb15-failing-for-your-errata-sheet/m-p/651047/highlight/true#M238402
    SET_BIT(PWR->UCPDR, PWR_UCPDR_UCPD_DBDIS);
    HAL_Delay(1);
#endif // STM32U5G9xx
```

Finally, you can start the debugger with this button: 

You can have a look at the port connected to the led LD1 (PORTB, pin 6) through the STM32CUBE REGISTERS TREE panel. The output port is the register called GPIO_ODR. If you select this register and click on it when the processor is stopped, the led will toggle.



Now, we will create a launch.json file.



We must choose the same debugger (STLink GDB server).

On this file we will add the liveWatch (in red) option like in the picture below.

```
"runEntry": "main",
"liveWatch": {
    "enabled": true,
    "samplesPerSecond": "4"
},
"imagesAndSymbols": [
```

With this live watch option, we can now have a look at any variable in the “STM32CUBE LIVE WATCH”.

Add for example the live watch below to see the led changing when the processor is running. This is the address of GPIOB->ODR and we want to see the bit 6 only.

```
((*(uint32_t*)0x42020414) >> 6) & 0x01
```

7. TASK TWO

We want to light the led corresponding to any pressed button (S1 to S4). It means for any button pressed, the corresponding led will be turned on, else it will be off. Just change your code to implement this behavior.

8. TASK THREE

The goal is to have the behavior described below:

- At start, the first led to the right will be turned on.
- Each time the BT1 button is pressed, the led to be lit moves to the left.
- Each time the BT4 button is pressed, the led to be lit moves to the right.
- If you are at a limit left and we press the button BT1, nothing should happen.
- Same behavior for the right with button BT4.

To have a nice programming method, you must create two arrays for leds descriptors

```
GPIO_TypeDef *ledPort[4]={ LD1_GPIO_Port, ... };
uint16_t ledBitNr[4]={LD1_Pin, ... };
```

HINT: the change must be made by pressing the button and not if the button is pressed. This means that you want to see the button status change.

9. TASK FOUR

Now we want to reflect the **number of buttons** pressed on a global variable called countButtons. This variable will be placed on the “Live Watch” window.

For example, if we press any 3 buttons, we want to see the value 3 in this variable. Write a function returning the value to display.

You must write a function who count the number of buttons pressed like:

```
uint16_t countPressedButtons(void)
```

As for the first laboratory, just display this value on the serial port.

Keep all you made in this laboratory for further laboratories.